

C++ STUDY NOTES

CHAPTER 4

Exception Handling and the I/O Stream Library

try/throw/catch, Stack Unwinding, Custom Exceptions, Stream Hierarchy, Formatted & Unformatted I/O, File Input and Output

University : Rani Channamma University, Belagavi

Course Code : 24MSCCS 1.3

Programme : M.Sc. Computer Science

Semester : First Semester

Syllabus : 2024-25 onwards

Author : Sudarshan Badli

Academic Yr : 2026-2027

AUTHORSHIP

Prepared and Created by: Sudarshan Badli

© 2026 Sudarshan Badli. All Rights Reserved.

Table of Contents

4.1 C-style Exception Handling	3
4.2 Terminating the Program; Checking Parameters Before the Function Call	4
4.3 C++-style Exception Handling (try, throw, catch)	5
4.4 Unwinding the Stack	6
4.5 Throwing Class Objects and Accessing the Thrown Object in the catch Block	8
4.6 Streams and the Stream Class Hierarchy	9
4.7 Unformatted Input-Output	10
4.8 Formatted Input-Output	11
4.9 File Input and Output	12
4.15 Examination Preparation — Chapter 4	15

HOW THIS CHAPTER IS ORGANISED

- Follows the official RCUB M.Sc. Computer Science syllabus (24MSCCS 1.3), Unit IV, exactly — no topic is skipped or compressed.
- Each topic: Definition → Syntax → Key points → Diagram/Table (where relevant) → Full worked C++ program(s) with output.
- Sections 4.10–4.14 add deep-dive material: dry run/trace table of stack unwinding, line-by-line explanation, advantages/disadvantages/applications, common mistakes, and head-to-head comparisons.
- Section 4.15 is a complete examination-preparation pack: 2/5/10-mark Q&A; programming questions, output prediction, find-the-error, MCQs and viva questions.

4.1 C-style Exception Handling

DEFINITION

Before C++'s built-in exception mechanism, C-style error handling relied on techniques such as checking a function's return value for an error code, using global flags like `errno`, or jumping with `setjmp/longjmp`. These approaches mix error-handling logic with normal logic and are easy to forget to check.

SYNTAX

```
// return-code style
int divide(int a, int b, int *result) {
    if (b == 0) return -1;    // error code
    *result = a / b;
    return 0;                // success
}
```

KEY POINTS

- Weaknesses of C-style handling: the caller can silently ignore the error code; error-handling code is scattered through the program; constructors cannot report failure this way (they have no return value).
- C++'s try/catch/throw mechanism (Topic 4.3) was designed to overcome these limitations.

PROGRAM

```
#include <iostream>
using namespace std;

int divide(int a, int b, int *result) {
    if (b == 0) return -1;
    *result = a / b;
    return 0;
}

int main() {
    int res;
    if (divide(10, 0, &res) == -1)
        cout << "Error: division by zero\n";
    else
        cout << "Result = " << res << "\n";
    return 0;
}
```

OUTPUT

```
Error: division by zero
```

4.2 Terminating the Program; Checking Parameters Before the Function Call

DEFINITION

Two complementary defensive strategies: (1) **validate parameters** before a risky function call actually executes, avoiding the error altogether; (2) when an unrecoverable error does occur, **terminate the program** in a controlled way using `exit()` or `abort()` rather than let it crash unpredictably or produce wrong results silently.

SYNTAX

```
if (!validInput(x)) {  
    cerr << "Invalid input\n";  
    exit(1);           // controlled termination, see Part C of the loops guide  
}  
result = riskyFunction(x); // only called once validated
```

KEY POINTS

- Checking parameters beforehand is cheaper than handling an exception afterwards and should be the first line of defence.
- `cerr` (unbuffered standard error stream) is conventionally used for error messages, separate from normal output (`cout`).
- `abort()` terminates immediately with no cleanup (raises `SIGABRT`); `exit(n)` terminates after flushing streams and running static destructors.

PROGRAM

```
#include <iostream>  
#include <cstdlib>  
using namespace std;  
  
double safeSqrtInput(double x) {  
    if (x < 0) {  
        cerr << "Error: negative input to sqrt-like function\n";  
        exit(1);           // controlled termination  
    }  
    return x;              // validated, safe to process further  
}  
  
int main() {  
    cout << safeSqrtInput(16) << "\n";  
    cout << safeSqrtInput(-4) << "\n"; // program terminates here  
    cout << "This line never runs\n";  
    return 0;  
}
```

OUTPUT

```
16  
Error: negative input to sqrt-like function
```

4.3 C++-style Exception Handling (try, throw, catch)

DEFINITION

C++ provides a structured exception mechanism using three keywords. Code that might fail is placed in a `try` block; when a problem is detected, `throw` raises an exception (any value or object); one or more `catch` blocks following the `try` block handle specific exception types, keeping error-handling logic cleanly separated from normal logic.

SYNTAX

```
try {
    // code that might throw
    if (errorCondition) throw someValue;
}
catch (ExceptionType1 e1) {
    // handle this type
}
catch (ExceptionType2 e2) {
    // handle this type
}
catch (...) {
    // catches ANY exception type not matched above
}
```

KEY POINTS

- Execution of the `try` block stops immediately at the `throw`; control jumps directly to the first matching `catch`.
- `Catch` blocks are checked in order, top to bottom; the first type match (including base-class matches) handles the exception.
- An exception that is thrown but never caught anywhere calls `std::terminate()`, ending the program.
- `catch ( ... )` (ellipsis) is a catch-all handler for any exception type, usually placed last.

PROGRAM – DIVISION WITH EXCEPTIONS

```
#include <iostream>
using namespace std;

double divide(double a, double b) {
    if (b == 0)
        throw string("Division by zero!"); // throw a string object
    return a / b;
}

int main() {
    try {
        cout << divide(10, 2) << "\n";
        cout << divide(5, 0) << "\n"; // throws here
        cout << "Never reached\n";
    }
    catch (string &msg) {
        cout << "Caught exception: " << msg << "\n";
    }
    cout << "Program continues normally\n";
    return 0;
}
```

OUTPUT

```
5
Caught exception: Division by zero!
Program continues normally
```

PROGRAM – MULTIPLE CATCH BLOCKS

```
#include <iostream>
using namespace std;

void check(int x) {
    if (x < 0) throw x;           // throws an int
    if (x == 0) throw string("zero"); // throws a string
    cout << "Valid positive number: " << x << "\n";
}

int main() {
    int values[] = {5, -3, 0};
    for (int v : values) {
        try { check(v); }
        catch (int e) { cout << "Caught int exception: " << e << "\n"; }
        catch (string &e) { cout << "Caught string exception: " << e << "\n"; }
    }
    return 0;
}
```

OUTPUT

```
Valid positive number: 5
Caught int exception: -3
Caught string exception: zero
```

4.
4

Unwinding the Stack

DEFINITION

Stack unwinding is the process that happens between a throw and the matching catch: the runtime system walks back up through every function call currently in progress, and for each one, properly destroys all local (automatic) objects that were fully constructed in that scope, calling their destructors, before control reaches the handler.

SYNTAX

```
void f3() { throw runtime_error("fail"); }
void f2() { Resource r; f3(); } // r is destroyed during unwinding
void f1() { f2(); }

int main() {
    try { f1(); } catch (exception &e) { ... }
}
```

KEY POINTS

- Stack unwinding guarantees that resources owned by local objects (file handles, memory, locks) are released even when an exception bypasses the normal return path — provided those resources are managed by objects whose destructors clean them up (RAII: Resource Acquisition Is Initialization).
- If a destructor itself throws during unwinding, the program calls `std::terminate()` — destructors should not throw.

PROGRAM

```
#include <iostream>
using namespace std;

class Resource {
    string name;
public:
    Resource(string n) : name(n) { cout << "Acquire " << name << "\n"; }
    ~Resource() { cout << "Release " << name << "\n"; }    // runs during unwinding
};

void inner() {
    Resource r("inner-resource");
    throw runtime_error("something failed");
}

int main() {
    try {
        Resource r("outer-resource");
        inner();
    }
    catch (exception &e) {
        cout << "Caught: " << e.what() << "\n";
    }
    return 0;
}
```

OUTPUT

```
Acquire outer-resource
Acquire inner-resource
Release inner-resource
Release outer-resource
Caught: something failed
```

KEY POINTS

- Both Resource destructors run automatically during unwinding, in reverse order of construction, before the catch block executes.

4. Throwing Class Objects and Accessing the Thrown Object in the catch Block

DEFINITION

An exception can be an object of any type, including a user-defined class — this lets an exception carry rich, structured information (an error code, a message, context data) rather than just a bare value. The standard library's `std::exception` hierarchy (`<stdexcept>`) provides ready-made exception classes with a virtual `what()` member that returns a description.

SYNTAX

```
class MyException {
    string message;
public:
    MyException(string m) : message(m) {}
    string getMessage() { return message; }
};

try { throw MyException("custom error"); }
catch (MyException &e) { cout << e.getMessage(); } // access the thrown object
```

KEY POINTS

- Catch the exception object by reference (`catch (MyException &e)`) to avoid an unnecessary copy and to allow polymorphic catching of derived exception types.
- Standard exceptions such as `std::runtime_error`, `std::out_of_range`, `std::invalid_argument` (all derived from `std::exception`) can be caught generically with `catch (exception &e)` and queried via `e.what()`.

PROGRAM – CUSTOM EXCEPTION CLASS

```
#include <iostream>
using namespace std;

class InsufficientFundsException {
    double shortBy;
public:
    InsufficientFundsException(double s) : shortBy(s) {}
    double getShortfall() { return shortBy; }
};

class Account {
    double balance;
public:
    Account(double b) : balance(b) {}
    void withdraw(double amt) {
        if (amt > balance)
            throw InsufficientFundsException(amt - balance); // throw a class object
        balance -= amt;
    }
};

int main() {
    Account acc(500);
    try {
        acc.withdraw(800);
    }
    catch (InsufficientFundsException &e) {
        cout << "Insufficient funds. Short by: " << e.getShortfall() << "\n";
    }
    return 0;
}
```

OUTPUT

```
Insufficient funds. Short by: 300
```

PROGRAM – STANDARD LIBRARY EXCEPTIONS

```

#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
    try {
        int arr[5] = {1,2,3,4,5};
        int idx = 7;
        if (idx >= 5)
            throw out_of_range("Index out of bounds");
        cout << arr[idx];
    }
    catch (exception &e) {           // catches ANY std::exception-derived type
        cout << "Standard exception: " << e.what() << "\n";
    }
    return 0;
}

```

OUTPUT

```
Standard exception: Index out of bounds
```

4.6 Streams and the Stream Class Hierarchy

DEFINITION

C++ I/O is built on the concept of a **stream**: an abstraction for a sequence of bytes flowing between a program and a source/destination (keyboard, screen, file, or a string in memory). All stream classes derive from a common base, `ios_base/ios`, giving them shared state (format flags, error state) while specialised classes add input or output behaviour.

SYNTAX

```

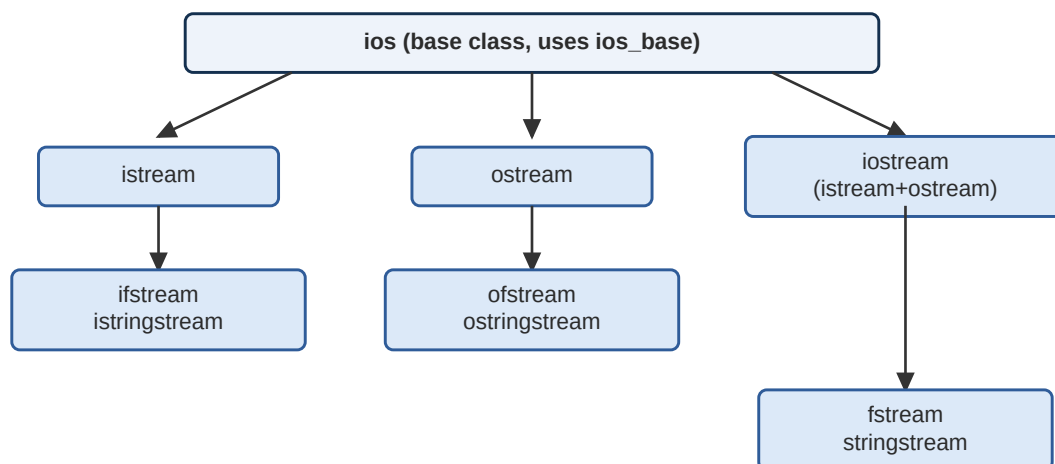
#include <iostream>    // cin, cout, cerr, clog
#include <fstream>     // ifstream, ofstream, fstream (files)
#include <sstream>     // istreamstring, ostreamstring, stringstream (strings)

```

KEY POINTS

- `cin` (standard input), `cout` (standard output), `cerr` (standard error, unbuffered), `clog` (standard error, buffered) are four predefined global stream objects.
- `istream` handles input (>>), `ostream` handles output (<<); `iostream` inherits from both for bidirectional streams.

STREAM CLASS HIERARCHY



PROGRAM

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";    // ostream (cout)
    cin >> age;                    // istream (cin)
    if (cin.fail()) {              // stream state check
        cerr << "Invalid input!\n"; // ostream (cerr)
    } else {
        cout << "You are " << age << " years old.\n";
    }
    return 0;
}
```

OUTPUT

```
Enter your age: 25
You are 25 years old.
```

4.7 Unformatted Input-Output

DEFINITION

Unformatted I/O transfers raw characters or blocks of bytes without interpreting them as a particular data type (no number parsing, no type-based formatting). Useful for reading whole lines, individual characters, or raw memory blocks.

SYNTAX

```
cin.get(ch);           // read a single character
cin.getline(buf, size); // read a line into a char array
cin.read(buf, n);       // read n raw bytes
cout.put(ch);           // write a single character
cout.write(buf, n);      // write n raw bytes
```

KEY POINTS

- `cin >>` stops at whitespace, so it cannot read a full sentence with spaces; `cin.getline()` reads until a newline (or the buffer limit), including spaces.
- `getline(cin, stringVar)` (the free function from `<string>`) is the modern, safer equivalent for `std::string`.

PROGRAM

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    char buffer[50];
    cout << "Enter a sentence: ";
    cin.getline(buffer, 50); // unformatted: reads the whole line
    cout << "You entered: " << buffer << "\n";

    char ch;
    cout << "Enter a character: ";
    cin.get(ch);
    cout.put(ch); // unformatted single-char output
    cout << "\n";
    return 0;
}
```

OUTPUT

```
Enter a sentence: C++ is powerful
You entered: C++ is powerful
Enter a character: Z
Z
```

4. Formatted Input-Output

DEFINITION

Formatted I/O controls how values are displayed — width, precision, number base, fill character, alignment — using either stream **manipulators** (from `<iomanip>`) or member functions of the stream object.

SYNTAX

```
#include <iomanip>
cout << setw(10) << setprecision(2) << fixed << value;
cout << setfill('*') << setw(8) << n;
cout << hex << n; // or oct, dec
```

Manipulator	Effect
<code>setw(n)</code>	Sets the field width for the next output item only
<code>setprecision(n)</code>	Sets number of significant digits / decimal digits
<code>fixed</code>	Displays floating-point values in fixed decimal notation
<code>setfill(ch)</code>	Sets the character used to pad unused field width
<code>left / right</code>	Sets alignment within the field width
<code>hex / oct / dec</code>	Changes the number base for integer I/O
<code>endl</code>	Inserts a newline AND flushes the stream buffer

Example

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14159265;
    cout << fixed << setprecision(2) << pi << "\n";           // 3.14
    cout << setw(10) << setfill('*') << 42 << "\n";           // *****42
    cout << hex << 255 << " " << oct << 255 << dec << "\n";     // ff 377
    return 0;
}
```

OUTPUT

```
3.14
*****42
ff 377
```

4. File Input and Output

DEFINITION

File I/O uses the stream classes `ofstream` (write), `ifstream` (read) and `fstream` (both), declared in `<fstream>`. A file stream must be **opened** before use and should be **closed** afterwards (or let its destructor close it automatically when it goes out of scope).

SYNTAX

```
ofstream outFile("data.txt");           // open for writing (creates/truncates)
outFile << "some text";
outFile.close();

ifstream inFile("data.txt");             // open for reading
string line;
while (getline(inFile, line)) { ... }
inFile.close();

fstream file("data.txt", ios::in | ios::out | ios::app); // combined modes
```

Mode flag	Meaning
<code>ios::in</code>	Open for reading
<code>ios::out</code>	Open for writing
<code>ios::app</code>	Append to the end of the file
<code>ios::ate</code>	Open and move to end immediately (can still seek)
<code>ios::trunc</code>	Discard existing file contents
<code>ios::binary</code>	Open in binary (not text) mode

Example – write then read a file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream outFile("marks.txt");           // write mode
    outFile << "Asha 88\n" << "Ravi 76\n";
    outFile.close();

    ifstream inFile("marks.txt");           // read mode
    string line;
    while (getline(inFile, line))
        cout << line << "\n";
    inFile.close();
    return 0;
}
```

OUTPUT

```
Asha 88
Ravi 76
```

4.10 Deep Dive — Exception Handling: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Separates error-handling code from normal program logic, making both easier to read.
- Cannot be silently ignored the way a return-code error can — an unhandled exception terminates the program loudly rather than letting bad data flow through unnoticed.
- Automatically unwinds the stack, correctly destroying local objects and releasing their resources (RAII) along the way.
- Allows a single handler to catch errors from deeply nested function calls without every intermediate function needing explicit error-checking code.

DISADVANTAGES

- Throwing and catching exceptions has runtime overhead compared with simple return-code checks, though only when an exception is actually thrown.
- Overusing exceptions for ordinary, expected control flow (rather than genuinely exceptional situations) can make code harder to follow.
- A destructor that itself throws during stack unwinding causes `std::terminate()` — destructors must never throw.
- Constructors have no return value, so exceptions are the only clean way to report that construction failed.

APPLICATIONS

- Reporting invalid user input, failed file operations, or network errors cleanly up the call stack.
- Resource management via RAII: a class's constructor acquires a resource, its destructor releases it, and exceptions guarantee that destructor still runs even if an error occurs mid-operation.
- Library code (including the C++ Standard Library) uses exceptions such as `std::out_of_range` and `std::bad_alloc` to report failures to calling code.

Line-by-Line Explanation

Line / Statement	Explanation
<code>double divide(double a, double b) {</code>	Begins a function that may fail (division by zero) and needs to signal that failure.

Line / Statement	Explanation
if (b == 0) throw string("Division by zero!");	Checks the risky condition; if true, throw immediately raises an exception carrying a string object, and the function stops executing right here — no return statement below it runs.
return a / b;	Runs only if b != 0; the normal, successful return path.
try {	Marks the start of a block whose statements might throw.
cout << divide(5, 0);	This call throws before divide() can return anything, so cout never receives a value to print here.
} catch (string &msg) {	Catches any thrown value whose type matches string (by reference, avoiding a copy); msg now refers to the thrown object.
cout << "Caught exception: " << msg;	Runs only if the matching catch above fired; prints the message carried by the exception.

COMMON MISTAKES / DEBUGGING TIPS

- Catching exceptions by **value** instead of by **reference** (catch (MyException e)) — this makes an unnecessary copy and, for a class hierarchy of exceptions, causes "slicing" (losing the derived part of a polymorphic exception object).
- Writing catch (. . .) first in a list of catch blocks — it matches everything, so any more specific catch blocks written after it become unreachable (dead code); the catch-all must always come last.
- Letting a destructor throw during stack unwinding — if this happens while another exception is already propagating, the program calls std::terminate() immediately.
- Forgetting to close an ofstream/ifstream explicitly in code paths that do not naturally reach the end of a function/block — prefer relying on the stream's own destructor (RAII) rather than a manual close() that might be skipped.
- Using cin >> variable to read a value that includes spaces (like a full name) — it stops at the first whitespace; use cin.getline() or getline(cin, str) instead.

4.11 Deep Dive — Dry Run: Stack Unwinding (Topic 4.4)

TRACE TABLE — ORDER OF EXECUTION

Step	Statement / Event	Effect
1	Resource r("outer-resource"); in main() try block	Constructor runs, prints "Acquire outer-resource"
2	inner() is called	Control enters inner()
3	Resource r("inner-resource"); in inner()	Constructor runs, prints "Acquire inner-resource"
4	throw runtime_error("something failed");	Exception raised; inner() stops executing immediately
5	Stack unwinding begins in inner()	inner-resource's destructor runs: prints "Release inner-resource"
6	Unwinding continues back into main()'s try block	outer-resource's destructor runs: prints "Release outer-resource"
7	Control reaches the matching catch block	Prints "Caught: something failed"

Notice that BOTH Resource destructors run — in reverse order of construction — before the catch block body executes at all. This guaranteed cleanup is the entire point of RAII combined with exceptions.

4.12 Comparison — C-style Error Handling vs C++ Exception Handling

Point	C-style (return codes / errno)	C++ (try / throw / catch)
Can be ignored by caller?	Yes — easy to forget to check	No — an unhandled exception terminates the program
Error data carried	Usually just an int code	Any type, including rich class objects

Point	C-style (return codes / errno)	C++ (try / throw / catch)
Works in constructors?	No (constructors have no return value)	Yes — the standard way to signal construction failure
Code separation	Error checks mixed into normal logic	Error handling isolated in catch blocks
Resource cleanup on error	Must be done manually at every failure point	Automatic via stack unwinding + RAII

4.13 Comparison — Formatted vs Unformatted I/O

Point	Unformatted I/O	Formatted I/O
Example	<code>cin.get(), cin.getline(), cout.put()</code>	<code>cout << setw(10) << setprecision(2) << x;</code>
Type interpretation	Raw characters / bytes, no parsing	Values interpreted and displayed per their type
Header needed	<code>iostream</code>	<code>iomanip</code> (for manipulators like <code>setw</code> , <code>setprecision</code>)
Typical use	Reading a full line including spaces	Aligning numbers in a table, controlling decimal places

4.14 Comparison — ifstream vs ofstream vs fstream

Class	Direction	Default open mode	Typical use
<code>ifstream</code>	Read only	<code>ios::in</code>	Reading an existing file
<code>ofstream</code>	Write only	<code>ios::out</code> (truncates existing content)	Creating / overwriting a file
<code>fstream</code>	Read and write	Must specify explicitly	Updating a file in place, or reading after writing

4.15

Examination Preparation — Chapter 4

NOTE ON QUESTION SOURCE

- The questions below are **expected / practice questions** prepared from the official RCUB syllabus for this chapter, in the common style of university exam papers.
- They are **not** claimed to be verified previous-year question papers — cross-check against your department's official previous-year papers for the exact verified questions, if available.

A. Two-Mark Questions and Answers

Q1. [2 marks] What is an exception?

An exception is a runtime problem or abnormal condition that disrupts the normal flow of a program, reported and handled using C++'s try/throw/catch mechanism.

Q2. [2 marks] List the three keywords used in C++ exception handling.

try (marks code that might fail), throw (raises an exception), catch (handles a specific exception type).

Q3. [2 marks] What happens if an exception is thrown but never caught?

The C++ runtime calls `std::terminate()`, which by default calls `abort()`, ending the program immediately.

Q4. [2 marks] What is stack unwinding?

The process of walking back up through active function calls between a throw and its matching catch, properly destroying (calling destructors on) local objects along the way.

Q5. [2 marks] What does `catch(...)` mean?

A catch-all handler that matches any exception type not matched by an earlier, more specific catch block; it must be written last in a catch chain.

Q6. [2 marks] Why should an exception be caught by reference rather than by value?

To avoid an unnecessary copy and to prevent "slicing" of polymorphic exception objects (losing derived-class information).

Q7. [2 marks] What is the purpose of `what()` in `std::exception`?

`what()` is a virtual member function that returns a C-string describing the exception, usable via `catch(exception &e;)` for any standard-library exception type.

Q8. [2 marks] Name two standard exception classes provided by .

`std::out_of_range` and `std::invalid_argument` (others include `std::runtime_error`, `std::logic_error`, `std::length_error`).

Q9. [2 marks] What is the difference between `cerr` and `clog`?

Both write to standard error; `cerr` is unbuffered (written immediately), while `clog` is buffered (may be delayed/flushed later).

Q10. [2 marks] What is the difference between `cin >>` and `cin.getline()`?

`cin >>` stops reading at the first whitespace; `cin.getline()` reads an entire line, including spaces, up to a newline or buffer limit.

Q11. [2 marks] Name the four predefined global stream objects in C++.

`cin` (standard input), `cout` (standard output), `cerr` (standard error, unbuffered), `clog` (standard error, buffered).

Q12. [2 marks] What is the use of the `setw()` manipulator?

`setw(n)` sets the output field width for the very next item only, useful for aligning columns of output.

Q13. [2 marks] What header is required to use `setprecision()` and `setw()`?

.

Q14. [2 marks] What is the difference between `ios::app` and `ios::trunc` file modes?

`ios::app` appends new data to the end of an existing file; `ios::trunc` discards (empties) the file's existing content before writing.

Q15. [2 marks] Why should a resource-owning object be preferred over manual `open()/close()` calls (RAII)?

Because the object's destructor automatically releases the resource (e.g. closes a file) even if an exception occurs, guaranteeing cleanup that a manual `close()` might otherwise skip.

B. Five-Mark Questions and Answers

Q1. [5 marks] Explain the try, throw and catch mechanism in C++ with a complete program.

Code that might fail is placed inside a try block; when a problem is detected, throw raises an exception, which can be any value or object; one or more catch blocks immediately following the try block handle specific exception types, keeping error-handling logic cleanly separated from normal logic. Execution of the try block stops the instant throw executes, and control jumps directly to the first catch block whose parameter type matches the thrown value, skipping every statement in between. The Topic 4.3 divide() example throws a string when dividing by zero, caught by catch (string &msg;), after which the program continues normally past the entire try/catch construct.

Q2. [5 marks] Explain stack unwinding with an example showing resource cleanup via destructors.

Stack unwinding is the process that occurs between a throw statement and its matching catch: the C++ runtime walks back up through every function call currently in progress, and for each one, destroys all local (automatic) objects that were fully constructed in that scope by calling their destructors, before control finally reaches the handler. This guarantees that resources owned by local objects (file handles, heap memory, locks) are released even when an exception bypasses the normal return path, provided those resources are managed by objects whose destructors clean them up — the RAII idiom. The Resource class example in Topic 4.4 demonstrates two Resource objects (outer and inner) both being destroyed, in reverse order of construction, during unwinding, before the catch block prints its message.

Q3. [5 marks] Discuss throwing class objects as exceptions, and explain how to access the thrown object's data in a catch block.

An exception need not be a bare value; it can be an object of a user-defined class, letting an exception carry structured information such as an error code, message, or other context. The thrown object is caught by reference (`catch (MyException &e;)`) to avoid an unnecessary copy and to allow polymorphic catching of derived exception types through a base class reference. The Topic 4.5 InsufficientFundsException example shows a class carrying a numeric shortfall, thrown from `Account::withdraw()` when funds are insufficient, and caught in `main()` where `e.getShortfall()` is called directly on the caught reference to retrieve and display the shortfall amount.

Q4. [5 marks] Explain the C++ stream class hierarchy with a diagram, and describe the role of cin, cout, cerr and clog. All C++ I/O is built around the concept of a stream, with every stream class ultimately deriving from `ios_base/ios`, which supplies shared state such as format flags and error flags. `istream` handles input (`>>`), `ostream` handles output (`<<`), and `iostream` inherits from both for bidirectional streams; `ifstream/ofstream/fstream` extend these for file I/O, and `istringstream/ostringstream/stringstream` extend them for in-memory string streams. `cin` is the predefined standard input stream object (an `istream`), `cout` the standard output stream (an `ostream`), `cerr` an unbuffered standard-error stream (writes immediately, useful for error messages that must appear right away), and `clog` a buffered standard-error stream (may be flushed later, suited to logging that is less urgent). The hierarchy diagram in Topic 4.6 shows how these relate.

Q5. [5 marks] Explain formatted I/O manipulators in C++ with a program showing `setw`, `setprecision`, `fixed` and `setfill`. Formatted I/O controls how values are displayed — field width, decimal precision, padding, alignment — using manipulators from or equivalent stream member functions. `setw(n)` sets the field width for only the next output item; `setprecision(n)` combined with `fixed` fixes the number of digits after the decimal point for floating-point output; `setfill(ch)` chooses the padding character used to fill unused width. The Topic 4.8 example shows `fixed << setprecision(2) << pi` printing "3.14" from 3.14159265, and `setw(10) << setfill('*') << 42` printing "*****42", demonstrating how these manipulators combine to produce neatly aligned, readable tabular output.

Q6. [5 marks] Explain file input and output in C++, including file modes, with a program that writes to and then reads from a file.

File I/O uses `ofstream` (write), `ifstream` (read) and `fstream` (both), declared in `<fstream>`; a file stream must be opened (either via its constructor with a filename, or an explicit `.open()` call) before use, and should be closed afterward (or allowed to close automatically via its destructor, RAII-style, when it goes out of scope). Open-mode flags such as `ios::in`, `ios::out`, `ios::app`, `ios::trunc` and `ios::binary` control exactly how the file is accessed; for example `ios::app` appends rather than overwriting, and `ios::binary` avoids text-mode translation for binary data. The Topic 4.9 example writes two lines to "marks.txt" using an `ofstream`, closes it, then reopens it with an `ifstream` and reads it back line by line with `getline()`, printing "Asha 88" and "Ravi 76".

C. Ten-Mark / Long-Answer Questions

Q1. [10 marks] Discuss exception handling in C++ comprehensively: motivation (compared with C-style error handling), the try/throw/catch mechanism, multiple catch blocks, stack unwinding, and throwing class objects, with complete programs.

Begin by contrasting C-style error handling (return codes, `errno`, `setjmp/longjmp`), which can be silently ignored by the caller and cannot report failure from a constructor, with C++'s structured try/throw/catch mechanism (Topics 4.1-4.3), which guarantees an unhandled exception terminates the program rather than letting incorrect results propagate silently. Explain the mechanics: a try block surrounds risky code, throw raises an exception of any type (a built-in value or a class object), and execution jumps immediately to the first catch block whose parameter type matches, skipping all intervening code; multiple catch blocks can handle different exception types from the same try block, checked top to bottom, with `catch(...)` as an optional, must-be-last, catch-all. Cover stack unwinding (Topic 4.4) in depth — the guaranteed destruction of local objects between the throw and the matching catch — and explain why this is the foundation of the RAII idiom for safe resource management. Conclude with throwing class objects (Topic 4.5), showing how a custom exception class can carry rich diagnostic data, accessed in the catch block via a reference parameter, and contrast this with catching standard-library exceptions (`std::exception` and its derivatives) via `e.what()`.

Q2. [10 marks] Explain the complete C++ stream class hierarchy and the distinction between unformatted and formatted I/O, with programs demonstrating both categories.

Present the stream hierarchy (Topic 4.6): `ios_base/ios` as the common base supplying shared format and error state; `istream` and `ostream` as the input/output specialisations; `iostream` combining both; and the file-stream (`ifstream/ofstream/fstream`) and string-stream (`istringstream/ostringstream/stringstream`) families built on top. Explain the four predefined global objects — `cin`, `cout`, `cerr` (unbuffered), `clog` (buffered) — and when each is appropriate. Unformatted I/O (Topic 4.7) transfers raw characters or bytes with no type-based interpretation — `cin.get()`, `cin.getline()`, `cin.read()`, `cout.put()`, `cout.write()` — essential for reading a full line including spaces, which `cin >>` cannot do since it stops at whitespace. Formatted I/O (Topic 4.8) instead controls how typed values are displayed, using manipulators such as `setw`, `setprecision`, `fixed`, `setfill`, `left/right` and `hex/oct/dec`; a complete answer should present at least one full worked program for each category, with traced output.

Q3. [10 marks] Discuss file handling in C++ in detail: file stream classes, opening modes, reading and writing text files, and error checking, with a complete program.

C++ file I/O (Topic 4.9) is handled through `ofstream` (write-only), `ifstream` (read-only) and `fstream` (read and write), all declared in `<fstream>`. A file stream is opened either via its constructor (`ofstream out("data.txt");`) or an explicit `open()` call, combined with mode flags such as `ios::in`, `ios::out`, `ios::app` (append rather than overwrite), `ios::ate` (seek to end but still allow seeking elsewhere), `ios::trunc` (discard existing content) and `ios::binary` (avoid text-mode translation). A thorough answer presents a complete program that opens a file for writing, writes several lines, closes it, then reopens the same file for reading and reads it back line by line using `getline(inFile, line)` in a while loop until end-of-file, printing the restored content — reproducing and explaining the Topic 4.9 marks.txt example. It

should also note good practice: checking a stream's state (e.g. `if (!inFile) {...}`) before relying on it, and preferring RAII (letting the stream's destructor close the file automatically) over manual `close()` calls that could be skipped if an exception occurs mid-operation.

Q4. [10 marks] Compare C-style and C++-style error handling in depth, and explain, with programs, how combining exception handling with RAII (via stack unwinding) provides safer resource management than manual cleanup.

C-style error handling (Topic 4.1-4.2) relies on techniques such as checking a function's return code, a global `errno` flag, or defensive parameter validation before a risky call, backed by `exit()/abort()` for unrecoverable situations; its core weakness is that a caller can simply fail to check the return code, letting an error silently propagate as if nothing went wrong, and it provides no way for a constructor (which has no return value) to report failure. C++ exception handling (Topic 4.3-4.4) instead makes errors impossible to silently ignore (an uncaught exception terminates the program) and, crucially, combines with stack unwinding to guarantee that every local object's destructor runs as control propagates up to the handler — this is the RAII idiom: a resource is acquired in a constructor and guaranteed to be released in the matching destructor, regardless of whether the function returns normally or via an exception. A strong answer presents the Resource class program from Topic 4.4 showing two levels of resource acquisition both being safely released during unwinding, and contrasts this with how equivalent manual-cleanup C-style code would need explicit, easy-to-forget cleanup code inserted at every possible early-exit point.

Q5. [10 marks] Write and explain a complete menu-driven C++ program for a simple file-based student record system that uses file I/O, formatted output, and exception handling together.

Design a program that lets the user add a student record (roll number, name, marks) to a text file using `ofstream` with `ios::app`, and display all records by reading the file back with `ifstream` and `getline()`, formatting the marks column with `setw()` and `setprecision()` for neat alignment (combining Topics 4.8 and 4.9). Wrap the file-opening logic in a try block, and throw a custom exception (or a standard one such as `runtime_error`) if the file cannot be opened, caught in `main()` with a message such as "Error: could not open student records file", demonstrating how exception handling, formatted I/O and file streams work together in one realistic, examinable program. The explanation should walk through why each `ifstream/ofstream` is allowed to close automatically via RAII rather than requiring an explicit `close()` in every branch, and why using exceptions here is preferable to checking a boolean return value from every file operation individually.

D. Important Programming Questions

PRACTICE: WRITE THE FULL PROGRAM

- Write a C++ program with a custom exception class `InvalidAgeException` thrown when a negative age is entered, caught and reported in `main()`.
- Write a C++ program demonstrating multiple catch blocks for `int`, `string` and a custom exception class, each thrown from different conditions in the same function.
- Write a C++ program that reads a list of numbers from a text file and prints their sum, average and maximum, formatted to 2 decimal places.
- Write a C++ program showing stack unwinding by creating two nested objects that print a message in their constructor and destructor, with an exception thrown inside the innermost function.
- Write a C++ program that copies the contents of one text file into another, handling the case where the source file does not exist using exception handling.

E. Output Prediction Questions

Q1. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    try {
        cout << "A ";
        throw 10;
        cout << "B ";           // never reached
    }
    catch (int e) {
        cout << "Caught " << e << " ";
    }
    cout << "C";
    return 0;
}
```

ANSWER

A Caught 10 C

Explanation: Execution stops the instant throw 10; fires, so "B " never prints; control jumps straight to the matching catch(int), then the program continues normally after the whole try/catch block.

Q2. Predict the output:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double val = 7.5;
    cout << setw(6) << setfill('0') << fixed << setprecision(1) << val;
    return 0;
}
```

ANSWER

0007.5

Explanation: setw(6) reserves a 6-character field; setfill('0') pads unused space with 0; fixed+setprecision(1) formats val as "7.5" (3 characters), so 3 zeros are added on the left to reach width 6: "0007.5".

Q3. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    try {
        try {
            throw string("inner");
        }
        catch (int e) {           // wrong type, does NOT match
            cout << "int handler";
        }
    }
    catch (string &e) {          // outer catch: matches
        cout << "outer caught: " << e;
    }
    return 0;
}
```

ANSWER

outer caught: inner

Explanation: The inner catch(int) does not match a thrown string, so the exception propagates OUT of the inner try block to the next enclosing try/catch, where catch(string&) correctly matches.

F. Find-the-Error Questions

Q1. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    try {
        throw 42;
    }
    catch (...) {
        cout << "caught something";
    }
    catch (int e) {           // ERROR: unreachable
        cout << "caught int: " << e;
    }
    return 0;
}
```

EXPLANATION

catch(...) is a catch-all and must always be the LAST handler in a catch chain. Placing it before catch(int e) makes the more specific handler unreachable (a compile error in standard C++, since catch(...) already matches everything).

Q2. Identify and correct the error:

```
#include <iostream>
using namespace std;

class MyError {
public:
    string msg;
};

void risky() { throw MyError{"failed"}; }

int main() {
    try {
        risky();
    }
    catch (MyError e) {       // works, but inefficient / can slice
        cout << e.msg;
    }
    return 0;
}
```

EXPLANATION

This compiles and runs, but catching by VALUE (MyError e) makes an unnecessary copy of the exception object and, for class hierarchies of exceptions, can "slice" off derived-class data. Best practice: catch (MyError &e;) (by reference).

Q3. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    ofstream outFile("data.txt"); // ERROR: missing header/std
    outFile << "hello";
    return 0;
}
```

EXPLANATION

ofstream requires #include (not just); without it, ofstream is an undeclared identifier and the program fails to compile.

G. Multiple Choice Questions (with explanations)

1. Which keyword raises (signals) an exception in C++?

(A) raise (B) throw ✓ (C) signal (D) error

Answer: (B). *throw* is the keyword used to raise an exception in C++ (unlike some other languages that use "raise").

2. What happens if a thrown exception is never caught by any catch block?

(A) The program ignores it and continues (B) `std::terminate()` is called, ending the program ✓ (C) The exception is converted to a return code (D) A compiler error occurs

Answer: (B). *An uncaught exception propagates to `std::terminate()`, which by default aborts the program.*

3. Which catch block, if present, must always be written LAST in a chain?

(A) `catch(int)` (B) `catch(string&)` (C) `catch(...)` ✓ (D) `catch(exception&)`

Answer: (C). *`catch(...)` matches any exception type, so placing it earlier would make later, more specific catch blocks unreachable.*

4. What is stack unwinding?

(A) Removing unused functions from code (B) Destroying local objects as control propagates from a throw to its catch ✓ (C) Reversing a stack data structure (D) Optimising recursive calls

Answer: (B). *Stack unwinding is the process of destroying local objects in each active function scope as the exception propagates up to a matching catch.*

5. Which member function of `std::exception` returns a description of the error?

(A) `message()` (B) `describe()` (C) `what()` ✓ (D) `getError()`

Answer: (C). *`what()` is the virtual member function, common to `std::exception` and its derivatives, that returns a C-string description.*

6. Which stream is unbuffered, writing output immediately?

(A) `cout` (B) `clog` (C) `cerr` ✓ (D) `cin`

Answer: (C). *`cerr` is unbuffered standard error output, written immediately; `clog` is buffered standard error.*

7. Which function reads an entire line, including embedded spaces, into a char array?

(A) `cin >>` (B) `cin.get(char)` (C) `cin.getline(buf, size)` ✓ (D) `cout.write()`

Answer: (C). *`cin.getline()` reads characters up to a newline (or the buffer limit), including spaces, unlike `cin >>` which stops at the first whitespace.*

8. Which header provides `setw()` and `setprecision()`?

(A) `iostream` ✓ (B) `iomanip` (C) `string` (D) `vector`

Answer: (B). *provides formatted-I/O manipulators like `setw()`, `setprecision()`, `setfill()`, fixed.*

9. Which file open mode appends to the end of an existing file rather than overwriting it?

(A) `ios::trunc` (B) `ios::in` (C) `ios::app` ✓ (D) `ios::ate`

Answer: (C). *`ios::app` opens the file so that all new writes go to the end, preserving existing content; `ios::trunc` instead discards it.*

10. What is the recommended way to catch a thrown class-type exception?

(A) By value (B) By pointer only (C) By reference ✓ (D) It cannot be caught, only built-in types can

Answer: (C). *Catching by reference (`catch (MyException &e;)`) avoids an unnecessary copy and prevents slicing of polymorphic exception types.*

H. Viva-Voce Questions

BE READY TO ANSWER VERBALLY, IN YOUR OWN WORDS

- Why can a return-code style error be dangerous compared with an exception?
- What is the lifetime relationship between a throw and the stack unwinding process?
- Why must `catch(...)` be the last handler in a chain?
- Can a constructor throw an exception? Can a destructor?
- What does RAII stand for, and how does it relate to exception safety?
- What is the difference between `cerr` and `clog` in terms of buffering?
- Why does `cin >>` fail to read a sentence containing spaces?
- What does `setw(n)` affect: just the next item, or all subsequent output?
- What is the difference in default behaviour between `ofstream` and an `fstream` opened with no explicit mode?
- Why is it good practice to let a file stream close via its own destructor rather than always calling `close()` manually?
- What does `ios::trunc` do, and how does it differ from `ios::app`?

- Why should an exception object generally be caught by reference, not by value?
- What is the role of `what()` in the `std::exception` hierarchy?
- If two catch blocks could both match an exception, which one runs?
- Give one real example where you would prefer exception handling over a simple if-check.

I. Chapter 4 — Revision Notes (Must-Remember Points)

TOP 10 EXAM-CRITICAL POINTS

- `try / throw / catch`: `try` surrounds risky code, `throw` raises an exception of any type, `catch` handles a matching type.
- Execution jumps straight from `throw` to the first matching `catch`, skipping everything in between.
- `catch(...)` is a catch-all and must be written LAST, or earlier specific catches become unreachable.
- Stack unwinding destroys local objects between `throw` and the matching `catch`, in reverse order of construction — the basis of RAII.
- Always catch class-type exceptions by reference (`catch (Type &e;)`), never by value, to avoid copies and slicing.
- `std::exception`'s `what()` returns a description string; standard exceptions include `out_of_range`, `invalid_argument`, `runtime_error`.
- Stream hierarchy: `ios_base/ios` base -> `istream/ostream` -> `iostream`; file streams `ifstream/ofstream/fstream`; string streams `istringstream/ostringstream/stringstream`.
- `cin >>` stops at whitespace; `cin.getline()` / `getline(cin,str)` read a full line including spaces.
- `setw(n)` affects only the NEXT output item; `setprecision()`+`fixed` together control decimal places.
- File modes: `ios::in` (read), `ios::out` (write), `ios::app` (append), `ios::trunc` (discard existing content), `ios::binary` (no text translation).